关节动作控制：启动、移动和停止

工具包 v0.3.0 · 2026-11-04 · 不依赖 Codex · Ubuntu/Linux

完整标定流程 · 各参数怎么测 · 真机配置模板 · 源码仓库

> **要实现的动作：**指定一个关节，从实测当前位置再转若干度，或到达指定角度；直线关节支持毫米。运动前后检查实际反馈，并让同一控制器组的其它轴保持采样时的位置。默认只生成预览，命令加 `--execute` 才执行。启动脚本只启动本工具，不自动使能电机、回零、释放抱闸或切换控制器。

1. 先运行一个无需硬件的例子

在完整工具包目录打开终端。以下生成一个有两个旋转轴和一个直线轴的 MuJoCo 示例；目录需要不存在，避免覆盖。

```
bash 控制关节.sh demo --destination /tmp/robotcal_motion_demo

# 从当前仿真角度再转 +5°，2秒完成：
bash 控制关节.sh move \
  --config /tmp/robotcal_motion_demo/mujoco.yaml \
  --joint joint1 --delta-deg 5 --duration 2 --execute

# 从当前仿真位置移到 -5°：
bash 控制关节.sh move \
  --config /tmp/robotcal_motion_demo/mujoco.yaml \
  --joint joint1 --target-deg -5 --duration 2 --execute

# 直线轴再走 +5毫米：
bash 控制关节.sh move \
  --config /tmp/robotcal_motion_demo/mujoco.yaml \
  --joint slider --delta-mm 5 --duration 2 --execute
```

有桌面环境时，在 move 命令末尾加 `--viewer` 打开 MuJoCo 查看器。去掉 `--execute` 只检查与预览。每次 MuJoCo 命令创建一个新的仿真实例，起始位置来自该配置；真机的相对运动则每次重新读取实际位置。示例参数仅用于程序验证，不是旭创或 OpenArm 的参数。

也可以统一从原入口调用：`bash 启动工具包.sh motion ...`，后面的 demo / inspect / move / stop 参数完全相同。程序内部按脚本位置和配置文件位置解析路径，不依赖 /home/tangyuan。完整离线包带 MuJoCo/Python；仅下载源码时先安装 pyproject.toml 的 mujoco 可选依赖。

2. 真机先具备什么，再启动什么

当前真机后端支持**已经运行并完成调试的 ROS 2 FollowJointTrajectory 控制器**。它不是任何机器人通用的底层电机驱动。若现场是 ROS 1、只有 CAN/EtherCAT SDK，或使用自定义消息，需要根据实际接口补充适配；不能仅填一个话题名字就声称接通。

1. 按原有流程启动实际机器人驱动和控制器。旭创目前尚未提供这条真实启动指令，本工具不猜启动包名、不自动启动已知 OpenArm 驱动代替它。
2. 在该机器人控制环境中 source 匹配的 ROS 和工作空间。下面的 Jazzy 是本机已测试版本；其它电脑按其实际安装填写。
3. 先运行 inspect，取得状态发布节点、实际状态话题、action 名称及 action 服务节点，核对它们确实连接本机硬件。
4. 填写工作范围、坐标定义、完整关节组、已验证的控制与停止行为，再做动作预览与受控实机执行。

```
source /opt/ros/jazzy/setup.bash
# 若实际机器人使用工作空间，再 source 它的 install/setup.bash。
```

```
bash 控制关节.sh inspect \
  --config configs/joint_motion_ros2.yaml \
  --ros-python /usr/bin/python3
```

ROS Python 与工具包离线 Python 分开使用，避免 ROS 的 Python ABI 与 NumPy/SDK 环境冲突。`--ros-python` 填安装 rclpy、sensor_msgs、control_msgs、trajectory_msgs 的解释器；也可设置环境变量 ROBOTCAL_ROS_PYTHON。启动脚本保留原 ROS 的 PYTHONPATH/动态库环境，再把控制请求交给这个解释器。不用 Codex 也可运行。

inspect 只发现接口和接收数据。`/joint_states` 的名称不证明它是真机反馈；要查发布者与原驱动。若状态发布来自 joint_state_publisher、GUI 或仿真，需要明确其来源，不能当独立编码器观测。

3. 配置填在哪里，哪些值不能猜

复制 `configs/joint_motion_ros2.yaml` 到现场项目的 configs 目录，再编辑。模板保留空值和 false；未知关节名、零位、范围不自动填成测试默认值。

字段	填写内容
backend / target	真实 ROS 接口填写 ros2 / real。protocol_test 只供隔离测试，要求 localhost、200—232测试域和 /robotcal_test/ 命名空间，不能用于普通真机控制器。
robot_id / urdf / coordinates_id	实际机器人标识、相对本配置的 URDF 路径、实际位置输入/命令的坐标定义版本。需确认身份、轴类型、方向、单位和工作范围；不表示几何精度已经完成标定。
joint_state_topic / state_publisher_node	实际状态话题及发布节点完整名称。检查时间戳和发布来源。当前 Jazzy 若不提供逐消息 publisher GID，则要求该话题只有一个发布者。
action_name / action_server_node	现有 FollowJointTrajectory action 及其服务器节点完整名称。不得猜测控制话题或向状态话题发送命令。
controller_joints	该控制器完整、有序、互不重复的独立关节列表。程序发送全组目标：选定轴运动，其余轴保持实测初始值。mimic 从动轴和 fixed 不能当作独立目标。
limits.每个关节	lower、upper、max_velocity、max_acceleration、path_tolerance、goal_tolerance、hold_tolerance、start_tolerance、stopped_velocity_tolerance、source。范围需包含在 URDF 中，速度不得超过 URDF；continuous 也必须给本次允许的有界多圈工作区。
confirmed	分别核实 model_and_joint_mapping、operating_limits、control_and_stop 后再填写 true；只改标记不能替代实际核对。
规程与接口来源	operating_procedure、commissioning_reference、motion_scope_reference、cancel_behavior_reference、exclusive_control_reference：实际项目规程、接口调试、当前姿态下允许的单轴运动区间、取消后的实际停止行为、独占控制交接依据。由项目建立，不要求存在整机厂家。
通信与时间	state_timeout_s、max_state_age_s、max_timestamp_skew_s、server_timeout_s、goal_time_tolerance_s、cancel_timeout_s、settle_s、point_interval_s、lead_time_s。按实际采集与网络能力配置。ROS_DOMAIN_ID/ 网络设置沿用实际系统。

单位：配置和 ROS 的位置为 rad/m，速度 rad/s 或 m/s，加速度 rad/s² 或 m/s²。CLI 的 deg/mm 只用于人工输入，程序转换一次。位置反馈和命令必须已经使用同一套关节坐标；若控制器输入仍为电机计数/电角度，应先实现和验证匹配的双向换算，不能只把 confirmed 改成 true。已有纯数据 feedback_mapping 不会自动改写硬件命令。

单轴上下界不能覆盖全部碰撞、线缆和闭环约束。motion_scope_reference 应明确本次其它关节姿态下允许的单轴区间；本控制入口没有通用碰撞规划器。是否可反驱、重力支撑和抱闸行为以实际装机设计为准。

4. 真机动作命令

以下将“实际关节名”替换为配置列表中的关节名；5°、2秒只是命令写法示例，实际值需落在已评估工况内。开始时读取实际位置并等待停稳，预览后执行还会重新采样；若起点变化超容差则拒绝，不自动归位。

```
# 预览：实际当前位置 +5°，暂不发送动作。
bash 控制关节.sh move \
  --config projects/旭创现场项目/configs/joint_motion_ros2.yaml \
  --joint 实际关节名 --delta-deg 5 --duration 2

# 执行同样的动作，日志目录需要不存在。
bash 控制关节.sh move \
  --config projects/旭创现场项目/configs/joint_motion_ros2.yaml \
  --joint 实际关节名 --delta-deg 5 --duration 2 \
  --execute --run-dir projects/旭创现场项目/results/MOVE001

# 到指定角度：把 --delta-deg 5 换成 --target-deg 5。
# 负方向：--delta-deg -5；直线轴：--delta-mm / --target-mm。
```

程序使用端点速度/加速度为零的五次轨迹，并检查整个轨迹的速度/加速度峰值。duration 过短时报告所需最短时间，不自动提高速度、裁剪目标或扩大限位。全组其它轴从实测值保持，不补零；检查发布反馈的时间戳、接收新鲜度、起点、跟踪误差和其它轴偏移。发现已有活动 goal 时拒绝抢占；同一电脑用户下的本工具副本共享控制器锁。不同电脑/其它客户端仍需项目的独占控制约定。

控制器结果成功后，程序还检查反馈确实到达目标并停稳。没有反馈、超范围、超速度、目标拒绝或控制器中止会留下失败记录。不会把“发出请求”或“action接受”当作动作成功。

5. 取消动作与停止核对

```
# 原动作正在运行时，在同一 ROS 环境的另一个终端执行：
bash 控制关节.sh stop \
  --run projects/旭创现场项目/results/MOVE001

# 动作终端也可以按 Ctrl+C，请求取消本次动作。
```

stop 从本次记录中读取非零 goal UUID，仅取消这个动作，禁止 cancel-all。先看取消回复，再等该 goal 的终态，并用实际反馈核对停稳。取消返回成功不自动等于停止已确认；反馈中断或控制器不响应时，报告 `stop_unconfirmed / stop_verified=false`，应采用现场既定停止方式。该取消接口不是硬件急停。程序不发送“回到零位”或抢发另一个目标来停止。

MuJoCo 的 stop 在原运行目录写入停止请求，由仿真控制循环制动并检查速度；查看原 result.json。仿真通常运行得很快，可加 --viewer 以实时速度观察；命令结束后该实例不再继续运行。

6. 输出怎么看，怎样用于标定

文件/状态	用途
request.json	本次操作、配置快照、原配置与 URDF 哈希及相对证据路径；外部 SDK 与主机锁是运行条件，不写死用户目录。
plan.json	实测起点、全组目标、轨迹和峰值；每次执行重新读取真实起点。
goal.json（ROS）	本工具本次 goal UUID、action、机器人ID、ROS域、接收/终态记录；stop 依赖它和 request.json。

feedback.jsonl / feedback.csv	ROS 实际位置/速度/测量时间/接收时间/发布者；MuJoCo 另记录目标与施加力。保留原始值，不自动标为已校准测量。
result.json / cancel.json	planned 只预览；completed 已按配置达到目标并停稳；stopped 本次停止经反馈确认；failed/blocked 未完成；stop_unconfirmed 停止情况未知。仿真/协议测试不能当作真机验收。

用于零位、限位、运动学采样时：原控制器到达姿态 → 本程序确认停稳 → 同步记录关节反馈和独立角度/位姿测量 → 标记 sample_id 与工况 → 换姿态。仅完成动作不证明关节坐标或末端精度；仍须独立测量和另采验证。

交付对应：D04 增加控制源码、启动脚本、配置与日志；D01 保存坐标/范围/接口来源；D05 保存实际动作和标定验证。只测试软件时明确保留“旭创真实接口未确认”的待完成项。

7. 来源与本次验证范围

- ROS 2 Joint Trajectory Controller：完整关节目标、容差、action结果。
- 官方 FollowJointTrajectory 定义：轨迹、误差码与结果。
- 官方 CancelGoal 定义：按特定 UUID 取消；不使用全零 UUID。
- 官方 JointState、rclpy ActionClient：位置消息与客户端API。
- MuJoCo 模型/状态/施加力：仿真通过 PD 力和 mj_step 积分，示例使用显式零重力/禁用接触配置。

轨迹数学、配置校验、运行记录、起点与停稳检查是本项目实现，不是 NVIDIA 官方点动工具。新增测试覆盖非零起点、角度/位移换算、其它轴保持、限制、旧结果保护和指定 UUID。已在本机 ROS 2 Jazzy 的隔离测试域验证只读预览、正常动作、拒绝、中止、反馈中断、其它轴偏移、Ctrl+C和主动取消；测试节点使用理想插值，没有连接硬件。MuJoCo 另做实际物理步进验证。旭创的底层驱动、真实节点名和装机工况仍需现场确认。

```
# 可重复执行的协议验证，输出目录需要不存在；只创建隔离的测试节点。
runtime/python/bin/python3 examples/verify_motion_ros2.py \
  --destination /tmp/robotcal_ros_motion_check --ros-python /usr/bin/python3
```